

Learning is a lifelong process of keeping your mind open to new ideas and experiences.

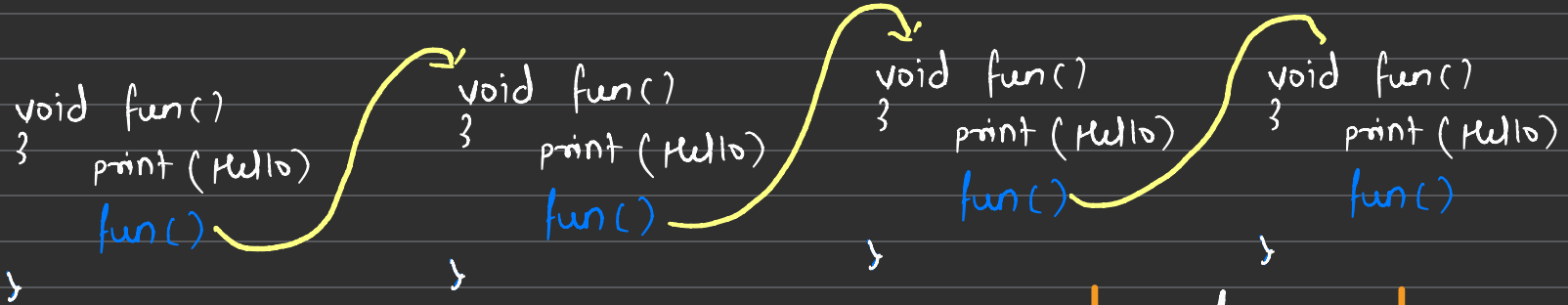
RECURSION & BACKTRACKING

- **RECURSIVE TREE**
- **DRY RUN**
- **FUNCTION CALL**



What is Recursion

Function calling itself until a specific condition is called as Recursion.



Hello Hello Hello Hello

```
main()
{
    fun()
}
```

Stack Overflow
Error

```
fun()
fun()
fun()
fun()
fun()
```

$cnt = 0$ ^{2/3}

```
void fun()  
{  
  if (cnt == 3)  
    return  
  cnt++  
  printf("Hello")  
  fun()  
}
```

```
void fun()  
{  
  if (cnt == 3)  
    return  
  cnt++  
  printf("Hello")  
  fun()  
}
```

```
void fun()  
{  
  if (cnt == 3)  
    return  
  cnt++  
  printf("Hello")  
  fun()  
}
```

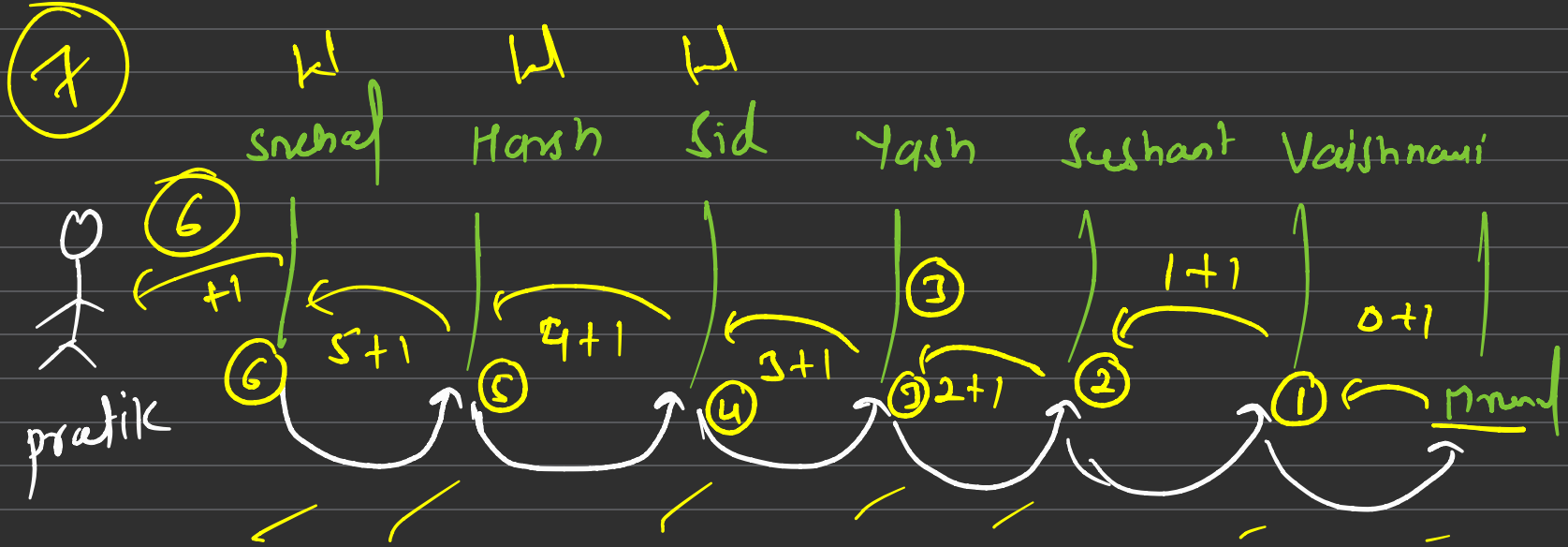
```
void fun()  
{  
  if (cnt == 3)  
    return  
  cnt++  
  printf("Hello")  
  fun()  
}
```

```
main()  
{  
  fun()  
}
```

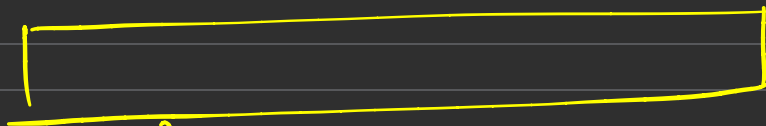
Base Case

Hello Hello Hello

Break large problem into smaller sub-problems

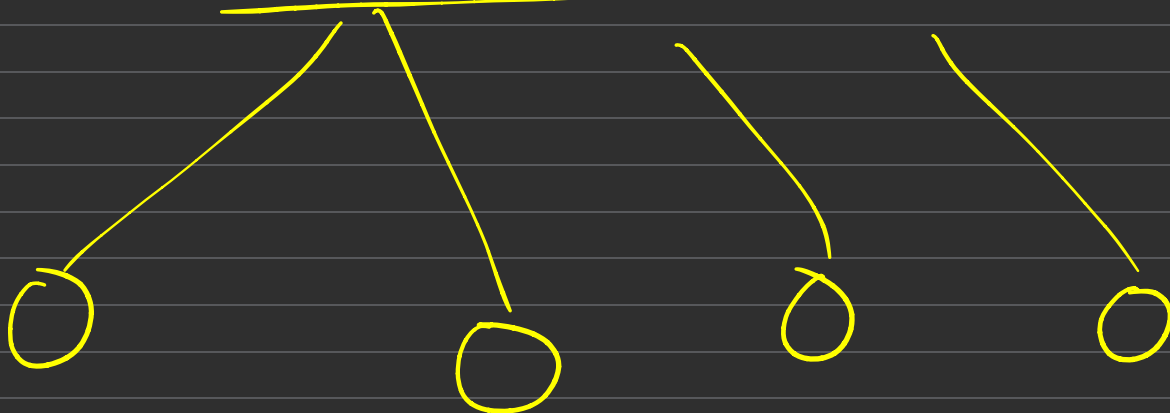


large

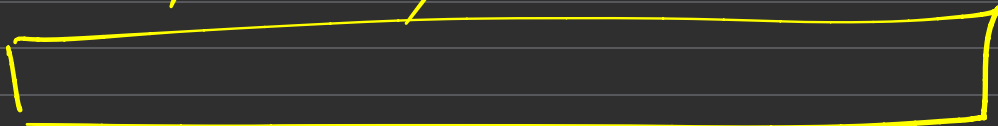


DnC

small



large



Activation Stack Record

```
void fun1()
{
    int x = 10;
    x++;
    print(x);
}
```

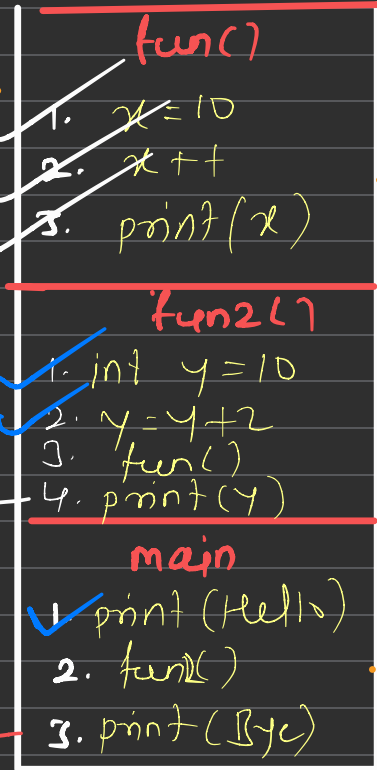
```
void fun2()
{
    int y = 10;
    y = y + 2;
    fun1();
    print(y);
}
```

```
main()
{
    print("Hello");
    fun2();
    print("Bye");
}
```

Hello 11 12 Bye

waiting

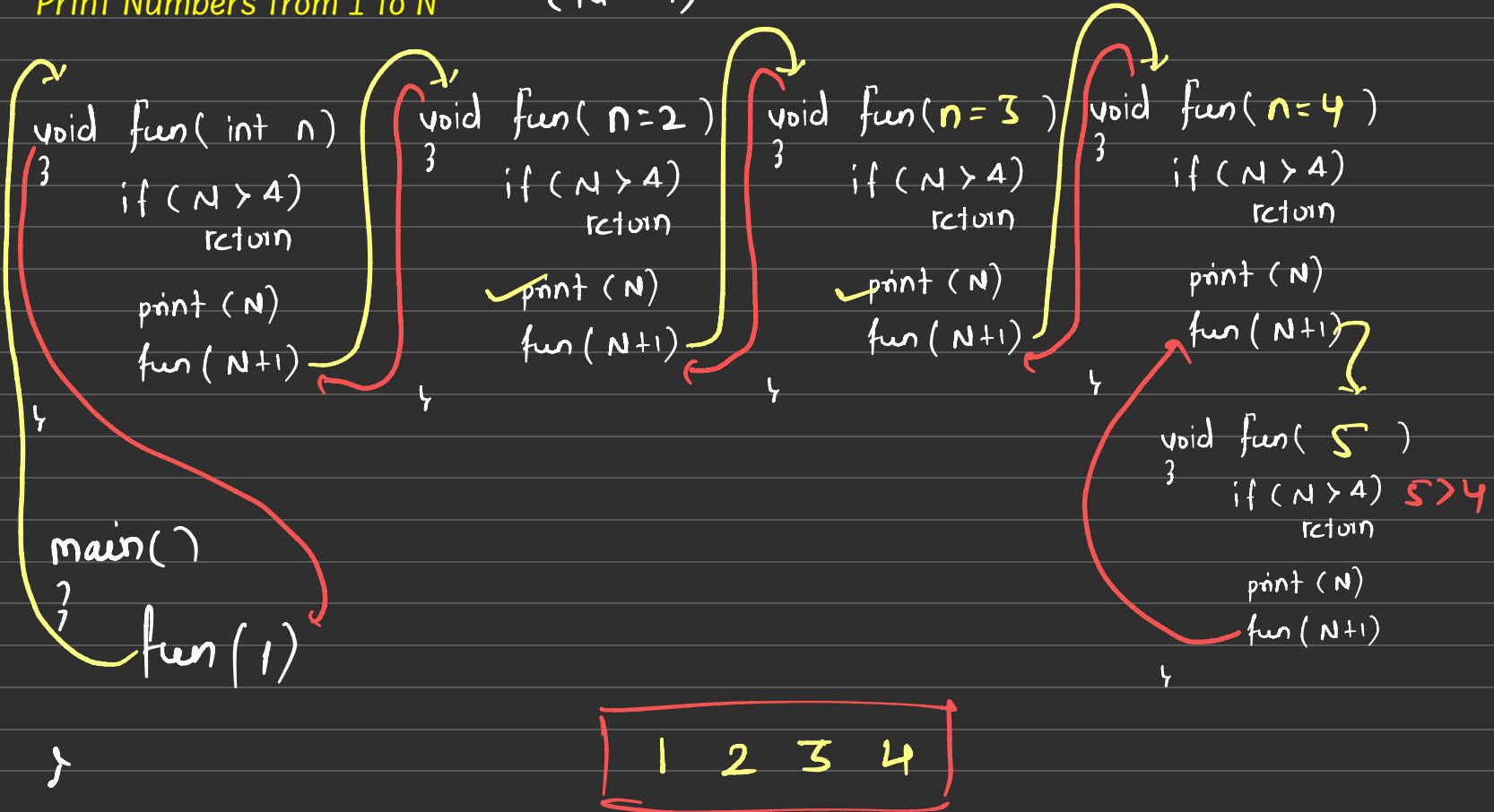
waiting



stack

Print Numbers from 1 to N

(N = 4)



Print Numbers from N to 1 (N=4)

```
void fun(n=4)
{
    if (n <= 0)
        return
    print(n)
    fun(N-1)
}
```

```
void fun(n=3)
{
    if (n <= 0)
        return
    print(n)
    fun(N-1)
}
```

```
void fun(int n)
{
    if (n <= 0)
        return
    print(n)
    fun(N-1)
}
```

```
void fun(int n)
{
    if (n <= 0)
        return
    print(n)
    fun(N-1)
}
```

```
void fun(int n)
{
    if (n <= 0)
        return
    print(n)
    fun(N-1)
}
```

```
main()
{
    fun(N)
}
```

4 3 2 1

N=4

print N to 1 But start from 1

```
void fun(1)
{
  if (N == 4)
    return
```

```
  fun(N+1)
  print(N)
}
```

```
void fun(2)
{
  if (N == 4)
    return
```

```
  fun(N+1)
  print(N)
}
```

```
void fun(3)
{
  if (N == 4)
    return
```

```
  fun(N+1)
  print(N)
}
```

```
void fun(4)
{
  if (N == 4)
    return
```

```
  fun(N+1)
  print(N)
}
```

```
main()
{
  fun(1)
}
```

3 2 1

Before

print (N)

fun (N+1)



print in Same
Order

After

fun (N+1)

print (N)

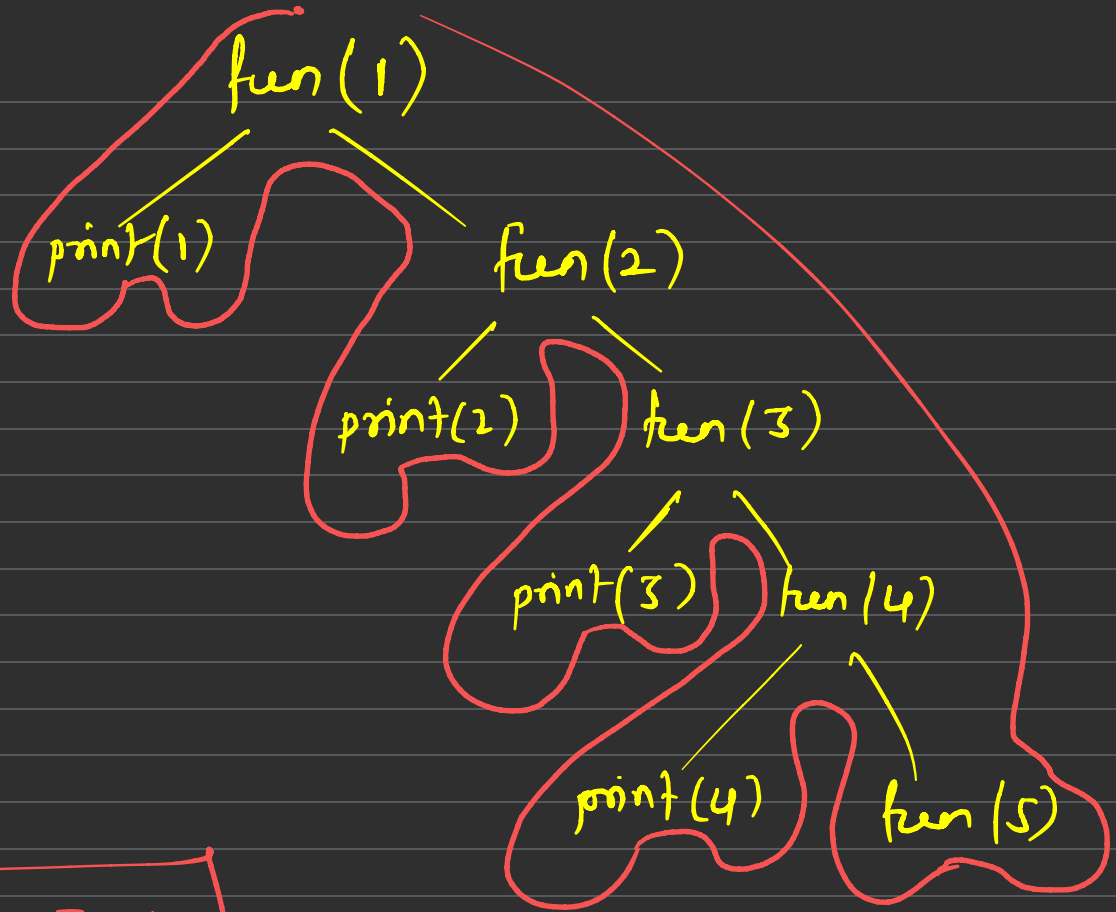


print in
Reverse order

Recursive Tree

```
void fun(N)
3  if (N > 4)
    return
    ✓ print(N)
    ✓ fun(N+1)
```

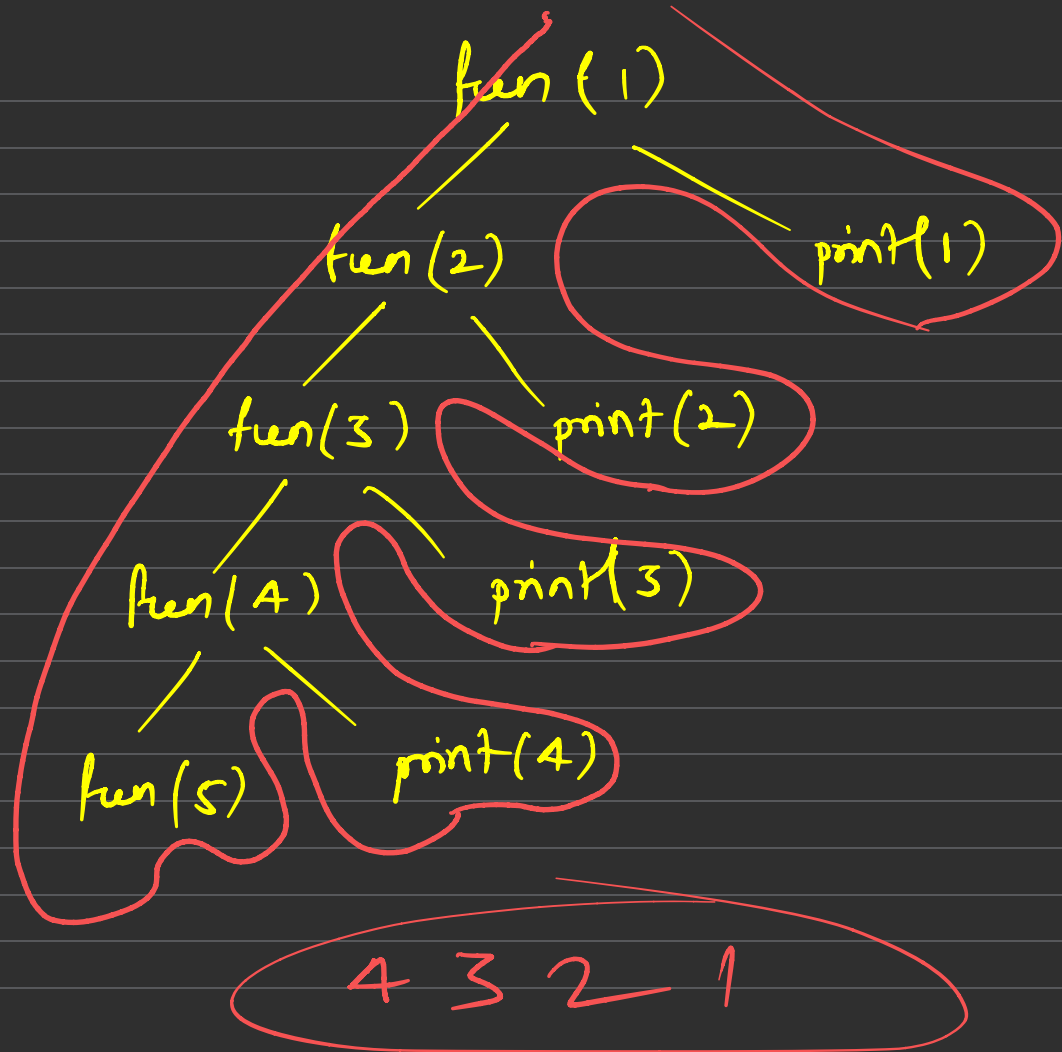
~



1	2	3	4
---	---	---	---

```
void fun(N)
{
    if (N == 5)
        return;

    fun(N+1);
    print(N);
}
```



Parameterised and Functional Recursion

Parameterised Recursion

Find the Sum of all the numbers from 1 to N

Functional Recursion

Find the Sum of all the numbers from 1 to N

Addition of digits in Number